

Real time monitor

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler
import time
```

```
WATCH_FOLDER = "sample_files"
```

```
class AVHandler(FileSystemEventHandler):
```

```
    def on_created(self, event):
```

```
        if event.is_directory:
```

```
            return
```

```
        print(f"\n[ALERT] New file detected: {event.src_path}")
```

```
        print("[INFO] Real-time protection triggered")
```

```
event_handler = AVHandler()
```

```
observer = Observer()
```

```
observer.schedule(
```

```
    event_handler,
```

```
WATCH_FOLDER,
recursive=True
)

observer.start()

print("=====")
print("Real-Time Protection Started")
print(f"Monitoring Folder: {WATCH_FOLDER}")
print("=====")

try:

    while True:
        time.sleep(1)

except KeyboardInterrupt:

    observer.stop()

observer.join()
Real time scanner
import hashlib
import json
import os
import shutil
```

```
import time
```

```
import yara
```

```
from watchdog.observers import Observer
```

```
from watchdog.events import FileSystemEventHandler
```

```
DATABASE_FILE = "malware_hashes.json"
```

```
QUARANTINE_FOLDER = "quarantine"
```

```
YARA_RULE_FILE = "yara_rules/malware_rule.yar"
```

```
WATCH_FOLDER = "sample_files"
```

```
def calculate_sha256(filepath):
```

```
    sha256 = hashlib.sha256()
```

```
    with open(filepath, "rb") as f:
```

```
        while chunk := f.read(4096):
```

```
            sha256.update(chunk)
```

```
    return sha256.hexdigest()
```

```
with open(DATABASE_FILE, "r") as f:
```

```
    database = json.load(f)
```

```
rules = yara.compile(filepath=YARA_RULE_FILE)
```

```
os.makedirs(QUARANTINE_FOLDER, exist_ok=True)
```

```
class AVHandler(FileSystemEventHandler):
```

```
    def on_created(self, event):
```

```
        if event.is_directory:
```

```
            return
```

```
        filepath = event.src_path
```

```
        print(f"\n[NEW FILE] {filepath}")
```

```
        try:
```

```
            file_hash = calculate_sha256(filepath)
```

```
            malware_detected = False
```

```
            detection_reason = ""
```

```
            # Hash Detection
```

```
            if file_hash in database:
```

```
malware_detected = True  
detection_reason = database[file_hash]
```

```
# YARA Detection
```

```
matches = rules.match(filepath)
```

```
if matches:
```

```
    malware_detected = True  
    detection_reason = f"YARA Rule: {matches[0]}"
```

```
if malware_detected:
```

```
    quarantine_path = os.path.join(  
        QUARANTINE_FOLDER,  
        os.path.basename(filepath)  
    )
```

```
    shutil.move(filepath, quarantine_path)
```

```
    print(  
        f"[MALWARE DETECTED] "  
        f"{filepath} -> {detection_reason}"  
    )
```

```
print(  
    f"[QUARANTINED] "  
    f"{quarantine_path}"  
)
```

```
else:
```

```
    print(f"[CLEAN] {filepath}")
```

```
except Exception as e:
```

```
    print(f"[ERROR] {e}")
```

```
event_handler = AVHandler()
```

```
observer = Observer()
```

```
observer.schedule(  
    event_handler,  
    WATCH_FOLDER,  
    recursive=True  
)
```

```
observer.start()
```

```
print("=====")
print("Next-Gen AV Real-Time Protection")
print(f"Monitoring: {WATCH_FOLDER}")
print("=====")
```

```
try:
```

```
    while True:
```

```
        time.sleep(1)
```

```
except KeyboardInterrupt:
```

```
    observer.stop()
```

```
observer.join()
```

```
Scanner
```

```
import hashlib
```

```
import json
```

```
import os
```

```
import shutil
```

```
from datetime import datetime
```

```
import yara
```

```
DATABASE_FILE = "malware_hashes.json"
```

```
REPORT_FILE = "scan_report.txt"
```

```
QUARANTINE_FOLDER = "quarantine"
```

```
YARA_RULE_FILE = "yara_rules/malware_rule.yar"
```

```
def calculate_sha256(filepath):
```

```
    sha256 = hashlib.sha256()
```

```
    with open(filepath, "rb") as f:
```

```
        while chunk := f.read(4096):
```

```
            sha256.update(chunk)
```

```
    return sha256.hexdigest()
```

```
def load_database():
```

```
    with open(DATABASE_FILE, "r") as f:
```

```
        return json.load(f)
```

```
def save_report(result):
```

```
    with open(REPORT_FILE, "a") as f:
```

```
        f.write(result + "\n")
```

```
database = load_database()
```

```
rules = yara.compile(filepath=YARA_RULE_FILE)
```



```
os.makedirs(QUARANTINE_FOLDER, exist_ok=True)
```

```
folder_path = input("Enter folder path to scan: ")
```

```
print("\nScanning Started...\n")
```

```
for root, dirs, files in os.walk(folder_path):
```

```
    for file in files:
```

```
        filepath = os.path.join(root, file)
```

```
        try:
```

```
            timestamp = datetime.now().strftime("%Y-%m-%d %H:%M:%S")
```

```
            file_hash = calculate_sha256(filepath)
```

```
            malware_detected = False
```

```
            detection_reason = ""
```

```
            # Hash Detection
```

```
            if file_hash in database:
```

```
                malware_detected = True
```

```
                detection_reason = database[file_hash]
```

```
# YARA Detection

matches = rules.match(filepath)

if matches:

    malware_detected = True

    detection_reason = f"YARA Rule: {matches[0]}"

if malware_detected:

    quarantine_path = os.path.join(
        QUARANTINE_FOLDER,
        os.path.basename(filepath)
    )

    shutil.move(filepath, quarantine_path)

    result = (
        f"[{timestamp}] MALICIOUS | "
        f"{filepath} | "
        f"{detection_reason} | "
        f"MOVED TO QUARANTINE"
    )

else:
```

```
result = (  
    f"[{timestamp}] CLEAN | "  
    f"{filepath}"  
)
```

```
print(result)
```

```
save_report(result)
```

```
except Exception as e:
```

```
error_msg = (  
    f"[ERROR] Failed to scan "  
    f"{filepath}: {e}"  
)
```

```
print(error_msg)
```

```
save_report(error_msg)
```

```
print("\nScan Completed")
```